Relatório Final — Etapa 2 — Oficina Mecânica

18 de outubro de 2026 · Grupo 7 — UCB

Integrantes: Davi Rodrigues Trida, Felipe Ricardo Silva, Fernando Rodrigues Cunha, Gabriel Wolney Drumond e Guilherme Comaccio

Laboratório de Banco de Dados (GPE17M40083) — UCB — Grupo 7. O banco da Etapa 1 foi revisado conforme a devolutiva e colocado em operação: aplicação web com perfis de acesso, transação de lançamento de peça, auditoria por gatilhos e estudo de desempenho com índices.

1. Identificação

Item	Valor
Tema	Oficina Mecânica
Integrantes	Felipe Ricardo Silva (analista de domínio), Fernando Rodrigues Cunha (modelador de dados), Gabriel Wolney Drumond (modelador lógico e normalização), Guilherme Comaccio (documentação), Davi Rodrigues Trida (administrador do banco)
Repositório	github.com/Felipe310202/Projeto_laboratorio_de_banco_de_dados
SGBD	MySQL 8 (testado em 8.0.46 e 8.4.11)
Aplicação	Python 3 + Flask, pasta `app/`

Como reconstruir o banco do zero. Rodar, nesta ordem: `sql/01_ddl.sql` (cria o banco), `02_carga.sql`, `03_consultas.sql` (opcional), `04_views.sql`, `05_triggers.sql`, `06_seguranca.sql` e a cópia local de `06b_usuarios_banco.exemplo.sql` com as senhas preenchidas. Os scripts `07_volume_teste.sql` e `08_indices.sql` são do estudo de desempenho (B4).

Como rodar a aplicação. Em `app/`: criar o ambiente (`python3 -m venv .venv`), instalar (`pip install -r requirements.txt`), copiar `.env.example` para `.env` com as senhas do banco e executar `python3 app.py`. Endereço: http://127.0.0.1:5000.

2. O que mudou desde a Etapa 1

Todos os pontos da devolutiva que dependem dos scripts foram corrigidos; o banco passou de 11 para 12 tabelas de negócio e de 4 para 17 restrições CHECK.

Critério	Ponto da devolutiva	Correção
C1	OS_PECA chamada de entidade	`status_historico` virou a entidade fraca:

Critério	Ponto da devolutiva	Correção
	fraca sem ser	identificada pela OS dona + chave parcial `num_sequencia`. OS_PECA e OS_MECANICO são associativas N:N com atributos
C1	Notação, totalidade da especialização, FKs no MER, layout	MER em revisão pela equipe: linha dupla (especialização total), "d" (disjunta), sem FKs no conceitual, uma só notação, novas entidades da seção 3
C2	Relações com PK e FK destacadas; 3FN/BCNF com chaves alternativas	Esquema reescrito (seção 3); `matricula` virou UNIQUE; dependência transitiva id_mecanico → cnpj_empresa → nome eliminada com a entidade `empresa_parceira`
C3	PKs nomeadas com `pk_`	12 PKs nomeadas
C3	CHECKs de domínio	Salário, preços, estoque, valor/hora, ano, previsão de entrega, status do histórico, formato de CPF/CNPJ
C4	Variedade de mecânicos e fornecedores	Mecânicos 8 → 18 (hierarquia de até 4 níveis), fornecedores 5 → 12, 4 empresas parceiras
C4	Q5 sem IN; Q6 sem "OS mais recente"	Q5 usa `IN` + `COALESCE`; Q6 filtra a OS mais recente de cada veículo
C5	Declaração de uso de IA; dicionário completo; README real	Declaração na seção 10 deste relatório; dicionário e README em atualização pela equipe com as tabelas novas

Outras correções encontradas pela equipe: o histórico de status não batia com o status atual em 35 das 47 OS (agora bate em todas); a garantia (RN14/RN15) não tinha suporte no modelo (entraram `problema_relatado` e o autorrelacionamento `id_os_garantia`); veículos gerados tinham marca e modelo incompatíveis ("Chevrolet Polo"); o `01_ddl.sql` agora cria o próprio banco.

3. Modelo lógico revisado

PK em **negrito**, FK em *itálico*, seguida da tabela referenciada.

- Cliente (**id_cliente**, nome, cpf_cnpj, telefone, email, endereco)
- Veiculo (**id_veiculo**, placa, modelo, marca, ano_fabricacao, *id_cliente*) — id_cliente → Cliente
- Mecanico (**id_mecanico**, nome, especialidade, *id_supervisor*) — id_supervisor → Mecanico
- MecanicoInterno (***id_mecanico***, matricula, salario, data_contratacao) — id_mecanico → Mecanico
- EmpresaParceira (**id_empresa**, nome, cnpj, telefone)
- MecanicoTerceirizado (***id_mecanico***, *id_empresa*, valor_hora_padrao) — id_mecanico → Mecanico; id_empresa → EmpresaParceira

- Fornecedor (**id_fornecedor**, nome, cnpj, telefone, endereco)
- Peca (**id_peca**, nome, descricao, *id_fornecedor*, preco_atual, estoque_atual) — id_fornecedor → Fornecedor
- OrdemServico (**id_os**, *id_veiculo*, problema_relatado, data_abertura, data_previsao_entrega, data_entrega_real, status_atual, *id_os_garantia*) — id_veiculo → Veiculo; id_os_garantia → OrdemServico
- StatusHistorico (***id_os***, **num_sequencia**, status, data_hora_mudanca, observacao) — id_os → OrdemServico (entidade fraca)
- OS_Peca (***id_os***, ***id_peca***, quantidade, preco_unitario_praticado) — id_os → OrdemServico; id_peca → Peca
- OS_Mecanico (***id_os***, ***id_mecanico***, horas_trabalhadas, valor_hora_praticado) — id_os → OrdemServico; id_mecanico → Mecanico

Normalização. Além das dependências a partir das PKs, valem cpf_cnpj → dados do cliente, placa → dados do veículo, matricula → dados do interno e cnpj → dados do fornecedor e da empresa. Todos esses determinantes são chaves candidatas (UNIQUE e NOT NULL), então não violam a BCNF. A única dependência transitiva do modelo anterior (cnpj_empresa → nome da empresa, dentro do terceirizado) foi eliminada com a relação EmpresaParceira. O esquema está em BCNF, com a desnormalização proposital de `preco_unitario_praticado` e `valor_hora_praticado` (RN10/RN19) mantida.

4. B1 e B2 — Aplicação e camada de acesso a dados

A aplicação web (Python + Flask) exige login e mostra a cada perfil só as telas que ele pode usar. Cada perfil conversa com o MySQL por uma conta própria do banco, então os GRANT/REVOKE da seção 7 valem também dentro da aplicação.

Requisito (B1)	Tela	Perfis
CRUD 1	Clientes (listar, buscar, cadastrar, editar, excluir)	atendente, gerente
CRUD 2	Veículos, com o dono escolhido numa lista	atendente, gerente
CRUD 3	Peças (consulta para todos; alteração só do gerente)	todos
CRUD 4	Mecânicos: grava em `mecanico` + subclasse na mesma transação	gerente
Consulta parametrizada 1	Faturamento por mecânico: período e tipo; JOIN de 3 tabelas + SUM/COUNT	gerente
Consulta parametrizada 2	Histórico do veículo: placa e período; JOIN + SUM por OS	gerente
Relatório por visão	Resumo das OS, lido da `vw_resumo_os`	atendente, gerente
Operação transacional	Lançar peça na OS (seção 5)	mecânico, gerente

Requisito (B1)	Tela	Perfis
Extra	Mudar status da OS (dispara gatilho) e tela de auditoria	todos / gerente

Tratamento de erro. A função `traduzir_erro` transforma o erro do driver em mensagem em português, pelo nome da restrição ou pelo código: CPF repetido (`uq_cliente_cpf_cnpj`) → "Já existe um cliente com esse CPF/CNPJ"; exclusão bloqueada por FK (1451) → "Este registro está em uso..."; gatilho da RN08 (1644) → "Um mecânico não pode ser supervisor de si mesmo"; falta de permissão (1142/1143) → "Seu perfil não tem permissão para esta operação".

Consultas parametrizadas. Todo SQL usa marcadores `%s` e os valores vão separados: `cur.execute("SELECT ... FROM cliente WHERE nome LIKE %s", (f"%{busca}%",))`. Nenhum valor digitado é concatenado no texto do comando.

Teste de injeção de SQL. Entrada `' OR '1'='1` na busca de clientes e no campo de login. Resultado: a busca devolveu 0 clientes e o login respondeu "Usuário ou senha inválidos". O texto foi tratado como valor literal, e não como parte do SQL.

Credenciais. Ficam em `app/.env`, listado no `.gitignore`; o repositório só tem o modelo `app/.env.example`, sem senhas.

Evidências: Figuras 1 a 8, no Anexo.

5. B3 — Transações e concorrência

Operação transacional: lançar peça na OS. Altera duas tabelas: insere o item em `os_peca`, com o preço do momento (RN10), e baixa o estoque em `peca`. O estoque só é baixado se houver quantidade suficiente (RN09); se não houver, tudo é desfeito.

```
conn.start_transaction()                                    # START TRANSACTION
cur.execute("INSERT INTO os_peca (id_os, id_peca, quantidade,
preco_unitario_praticado) "
            "SELECT %s, id_peca, %s, preco_atual FROM peca WHERE id_peca =
%s",
            (id_os, qtd, id_peca))
cur.execute("UPDATE peca SET estoque_atual = estoque_atual - %s "
            "WHERE id_peca = %s AND estoque_atual >= %s", (qtd, id_peca,
qtd))
if cur.rowcount == 0:      # estoque insuficiente
    conn.rollback()                                         # ROLLBACK
else:
    conn.commit()                                           # COMMIT
# qualquer erro do banco no meio -> conn.rollback()
```

Cenário	Entrada	Resultado
Sucesso	OS 13, filtro de óleo, 3 unidades	INSERT 1 linha, UPDATE 1 linha, COMMIT; estoque 60 → 57
Falha provocada	OS 13, kit embreagem, 10	INSERT 1 linha, UPDATE 0 linha, ROLLBACK; nenhum

Cenário	Entrada	Resultado
(aplicação)	unidades (estoque 6)	item gravado, estoque continua 6
Falha provocada (Workbench)	Mesmo lançamento, com UPDATE sem a condição de estoque	O INSERT aparece dentro da transação; o UPDATE falha com erro 3819 (`ck_peca_estoque`); após o ROLLBACK o item some e o estoque continua 6

Nos dois casos de falha o banco permaneceu íntegro: nem a OS cobrou peça que não saiu do estoque, nem o estoque mudou.

Evidências: Figuras 9 e 10, no Anexo.

Experimento de concorrência (duas sessões simultâneas sobre a mesma peça, nível REPEATABLE READ, executado em ambiente de teste com MySQL 8.0.46 e o mesmo banco do projeto):

Experimento	Sessão A	Sessão B	Comportamento observado
Bloqueio e espera	`START TRANSACTION; UPDATE peca ... WHERE id_peca = 1;` e espera 3 s antes do COMMIT	`UPDATE peca ... WHERE id_peca = 1;`	B ficou bloqueada 2,0 s e só executou depois do COMMIT de A
Tempo de espera esgotado	Segura a linha da peça 1 por 6 s	Mesmo UPDATE com `innodb_lock_wait_timeout = 3`	B recebeu erro 1205 "Lock wait timeout exceeded" após 3 s
Deadlock	Trava a peça 1 e depois tenta a peça 2	Trava a peça 2 e depois tenta a peça 1	O InnoDB detectou o impasse, desfez a transação de B com erro 1213 "Deadlock found" e deixou A concluir; `SHOW ENGINE INNODB STATUS` registrou "WE ROLL BACK TRANSACTION (2)"

Nível de isolamento adotado: REPEATABLE READ (padrão do MySQL, conferido com `SELECT @@transaction_isolation`). Justificativa: a disputa real da oficina é por estoque, e ela é resolvida pelo bloqueio de linha do próprio UPDATE condicional (`estoque_atual >= qtd`), que serializa dois lançamentos da mesma peça sem precisar de SERIALIZABLE, que bloquearia também as leituras e reduziria a concorrência nas telas de consulta. READ COMMITTED também funcionaria para o lançamento, mas o REPEATABLE READ dá leituras estáveis dentro de uma transação (relatórios coerentes) sem custo perceptível neste volume. Para reduzir deadlocks, a aplicação lança uma peça por transação, o que mantém uma ordem única de bloqueio.

6. B4 — Otimização de consultas

Dois dos três índices trouxeram ganho claro; o terceiro só ajuda quando a busca não começa com curinga. Volume usado: 100.047 OS, 300.117 itens de peça, 100.067 lançamentos de mão de obra e 5.045 clientes (`07_volume_teste.sql`), MySQL 8.0.46. Com as 47 OS da carga normal, o otimizador lê a tabela inteira mais rápido do que usaria um índice, por isso o volume foi aumentado só para o estudo. Tempos medidos com `EXPLAIN ANALYZE`.

Consulta	Antes: plano	Antes: tempo	Índice criado	Depois: plano	Depois: tempo
1. OS "aguardando peca" abertas em junho/2026	`type ALL`, 99.875 linhas lidas	36,3 ms	`idx_os_status_data (status_atual, data_abertura)`	`range` no índice, 411 linhas, índice de cobertura	0,69 ms
2. Faturamento por mecânico em junho/2026 (JOIN + GROUP BY)	`type ALL` em ordem_servico (100.047 linhas) + tabela temporária	62,5 ms	`idx_os_data_abertura (data_abertura)`	`range` com 4.124 OS + busca pela PK em os_mecanico	25,1 ms
3a. Cliente com nome `LIKE '%Teste 12%'`	`type ALL`, 5.045 linhas	2,66 ms	`idx_cliente_nome (nome)`	`index` (varre o índice inteiro), 5.045 linhas	1,89 ms
3b. Cliente com nome `LIKE 'Carla Teste 1%'`	`type ALL`, 5.045 linhas	1,28 ms	mesmo índice	`range`, 139 linhas	0,09 ms

Leitura dos planos. `type ALL` significa leitura da tabela inteira; `range` significa que o MySQL foi direto a um intervalo do índice. Na consulta 1, "Using index" indica índice de cobertura: a resposta sai só do índice, sem ler a tabela.

Ordem das colunas. No índice da consulta 1, `status_atual` vem primeiro porque é filtro de igualdade, e `data_abertura` depois porque é intervalo; na ordem inversa o índice só aproveitaria a data. Esse mesmo índice não serve à consulta 2, que não filtra status, por isso ela ganhou um índice só pela data.

Ganho parcial na consulta 2. O tempo caiu de 62,5 para 25,1 ms, menos que na consulta 1, porque o custo restante está na junção (4.124 buscas pela PK de os_mecanico) e na agregação em tabela temporária, que o índice não elimina.

Resultado negativo (consulta 3a). Com curinga no início (`'%texto%'`), o índice B-tree não sabe por onde começar e o MySQL varre o índice inteiro: o ganho foi desprezível (2,66 → 1,89 ms). O índice só vale para busca pelo começo do nome (3b). Para busca no meio do texto, a alternativa seria um índice FULLTEXT. Os índices estão em `sql/08_indices.sql`.

7. B5 — Segurança e controle de acesso

Quatro papéis (roles) do MySQL aplicam o menor privilégio, e cada conta do banco recebe um deles (`sql/06_seguranca.sql`). As contas e suas senhas ficam fora do Git: o repositório tem só o modelo `06b_usuarios_banco.exemplo.sql`.

Papel	Pode	Não pode
`papel_login`	Ler a tabela `usuario` para conferir a senha	Qualquer outra coisa
`papel_atendente`	Clientes, veículos e OS sem excluir; consultar peças, garantias e o resumo; ver mecânicos pela `vw_mecanico_publico`	Ver salário; alterar peças (REVOKE); excluir; ver auditoria
`papel_mecanico`	Ver OS pela `vw_os_oficina`; lançar peças e horas; mudar status; baixar estoque (privilégio só na coluna `estoque_atual`)	Ver clientes; mudar preço; ver auditoria
`papel_gerente`	Todas as tabelas de negócio, salários e usuários; ler a auditoria	Apagar ou alterar a auditoria

REVOKE com motivo. O atendente começou com `SELECT, UPDATE` em `peca`; o UPDATE foi revogado porque o estoque só deve mudar pela transação de lançamento (RN09) e o preço só pelo gerente, com auditoria.

Visões de restrição. `vw_mecanico_publico` esconde colunas (salário, matrícula, data de contratação, valor/hora). `vw_os_oficina` esconde linhas (só OS em andamento) e colunas (nenhum dado do cliente).

Testes com login real. Atendente: viu 45 clientes e 18 mecânicos pela visão; recusado ao ler salário e ao excluir cliente (erro 1142). Mecânico: viu 12 OS pela visão; recusado ao ler clientes (1142) e ao mudar preço (1143, privilégio por coluna). Gerente: leu salários e auditoria; recusado ao apagar a auditoria (1142).

Senhas da aplicação. A tabela `usuario` guarda só o hash bcrypt (custo 12, sal aleatório embutido), ex.: `$2b$12$cfPd454qVAq4...`. Um CHECK recusa qualquer valor que não tenha formato de hash bcrypt, o que impede gravar senha em texto puro por engano.

Dados pessoais (LGPD). São dados pessoais: nome, CPF, telefone, e-mail e endereço do cliente; placa (vinculada a uma pessoa); nome, matrícula, salário e data de contratação do mecânico; login dos usuários. Cuidados adotados: mecânico não tem acesso nenhum à tabela de clientes; salário só visível ao gerente; senhas só em hash; credenciais fora do repositório; carga só com dados fictícios.

Evidências: Figuras 11 a 14, no Anexo.

8. B6 — Gatilhos e auditoria

A tabela `auditoria` registra tabela, registro, campo, valor anterior, valor posterior, autor e data/hora. Dois gatilhos de auditoria e dois de regra estão em `sql/05_triggers.sql`.

Gatilho	Evento	O que faz
`trg_peca_audit_preco`	AFTER UPDATE em `peca`	Registra a mudança de `preco_atual`; ignora mudança só de estoque
`trg_os_audit_status`	AFTER UPDATE em `ordem_servico`	Registra a mudança de status e grava o próximo evento no `status_historico` (RN16), calculando o `num_sequencia`
`trg_mecanico_rn08_ins` / `_upd`	BEFORE INSERT / UPDATE em `mecanico`	Cancela com SIGNAL se o mecânico for supervisor de si mesmo (RN08)

Por que auditar preço e status. O preço define o que o cliente paga nas próximas OS; um reajuste errado precisa de autor e data. O status mostra onde está o carro e se o prazo foi cumprido; numa reclamação, a oficina precisa saber quem mudou e quando.

Autor. A aplicação usa uma conta do banco por perfil, então informa a pessoa logada com `SET @usuario_app = 'login'` antes de alterar; sem isso, o gatilho grava `USER()`.

Resultado. A troca de preço do filtro de óleo (peça 2) de 40,00 para 38,00 pelo gerente gerou o registro "peca 2, preco_atual, 40.00 → 38.00, sebastiao.gerente" (a linha anterior, 35.00 → 40.00, é de um teste feito antes pelo administrador). A mudança da OS 13 de aberta para concluída gerou o registro na auditoria e um novo evento no histórico de status. Uma alteração desfeita com ROLLBACK não deixou registro, porque o gatilho roda dentro da mesma transação (Figura 15).

Evidência: Figura 15, no Anexo.

9. B7 — Projeto físico e armazenamento

Revisão dos tipos de dados.

Campo	Tipo atual	O que mudaríamos	Por quê
`cpf_cnpj`, `cnpj`	VARCHAR(14) + CHECK de dígitos	CHAR(14) para CNPJ; manter VARCHAR só no cliente (11 ou 14)	Tamanho fixo e conhecido
`status_atual`, `status`	VARCHAR(20) + CHECK	Tabela de domínio `status_os` com FK	Evita repetir a lista de 5 valores em dois CHECKs e permite rótulo com acento
`ano_fabricacao`	INT	YEAR ou SMALLINT	2 bytes em vez de 4
`horas_trabalhadas`	DECIMAL(5,2)	Manter	Até 999,99 h por OS, suficiente
`placa`	VARCHAR(8)	CHAR(7) com CHECK do	Placa tem sempre 7

Campo	Tipo atual	O que mudaríamos	Por quê
		padrão Mercosul	caracteres
`telefone`	VARCHAR(20)	VARCHAR(15) com CHECK de dígitos	Hoje aceita qualquer texto

Crescimento em 2 anos. Hipótese: 15 OS por dia útil, 300 dias por ano, 2,5 peças e 1,4 mecânico por OS, 4 eventos de status por OS.

Tabela	Linhas em 2 anos	Tamanho estimado
ordem_servico	9.000	~1,5 MB (dados + índices)
os_peca (maior movimento)	22.500	~1,5 MB
os_mecanico	12.600	~0,8 MB
status_historico	36.000	~2 MB
auditoria	~27.500	~3 MB

Tamanhos calculados a partir da medição real do teste de volume (100 mil OS ocupam 15,6 MB entre dados e índices). O banco inteiro deve ficar abaixo de 20 MB em 2 anos.

Plano de backup.

- **O que:** o banco `oficina_mecanica` inteiro, com gatilhos e visões: `mysqldump --single-transaction --routines --triggers oficina_mecanica`. O `--single-transaction` tira uma cópia consistente sem travar as tabelas InnoDB.
- **Frequência:** completo toda noite às 23h, depois do expediente; log binário ativo para recuperar até o minuto da falha.
- **Retenção:** 7 diários, 4 semanais e 12 mensais; uma cópia fora da máquina do servidor.
- **Teste de restauração:** uma vez por mês, restaurar o último dump num banco temporário e conferir as contagens das tabelas principais e o total da `vw_resumo_os`.

Arquivamento. O volume previsto não justifica particionamento. As tabelas que crescem sem parar são `status_historico` e `auditoria`; a proposta é mover, uma vez por ano, os registros com mais de 2 anos para `status_historico_arquivo` e `auditoria_arquivo`. Particionamento nativo não seria a escolha aqui porque o MySQL não permite tabela particionada com chave estrangeira, e `status_historico` depende de `ordem_servico`.

10. Declaração de uso de IA

A equipe usou o Claude (Anthropic) como apoio nas seguintes partes: análise da devolutiva da Etapa 1; reescrita dos scripts `01` a `03`; código da aplicação Flask e roteiro dos testes de transação e concorrência

Como o resultado foi verificado. As decisões de modelo (empresa parceira como entidade, suporte à garantia) foram tomadas pela equipe. Cada script foi executado do zero no MySQL 8.4.11 do integrante administrador do banco, com conferência das contagens; as visões, os gatilhos e os perfis foram

testados com consultas e com login real em cada conta; a aplicação foi executada localmente e cada tela foi testada nos três perfis. Os integrantes revisaram o conteúdo para conseguir explicá-lo na arguição.

Anexo — Figuras

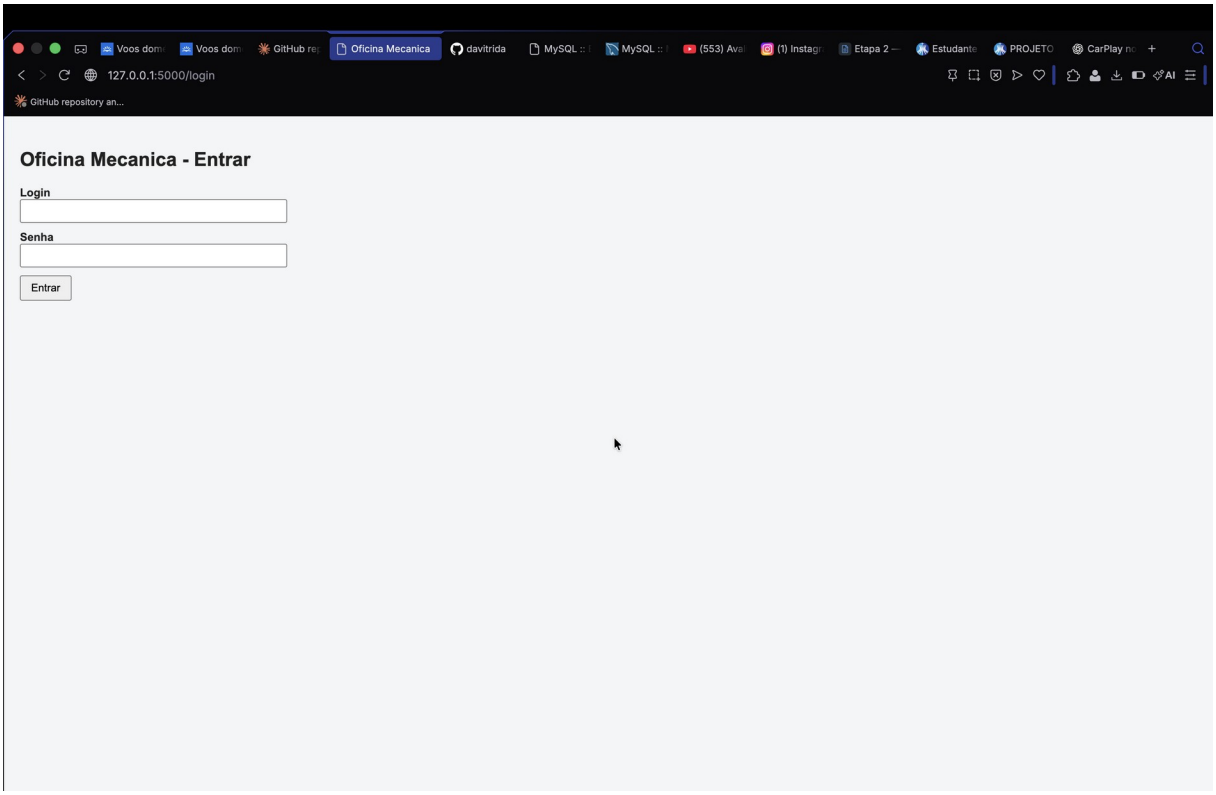


Figura 1 — Tela de login.

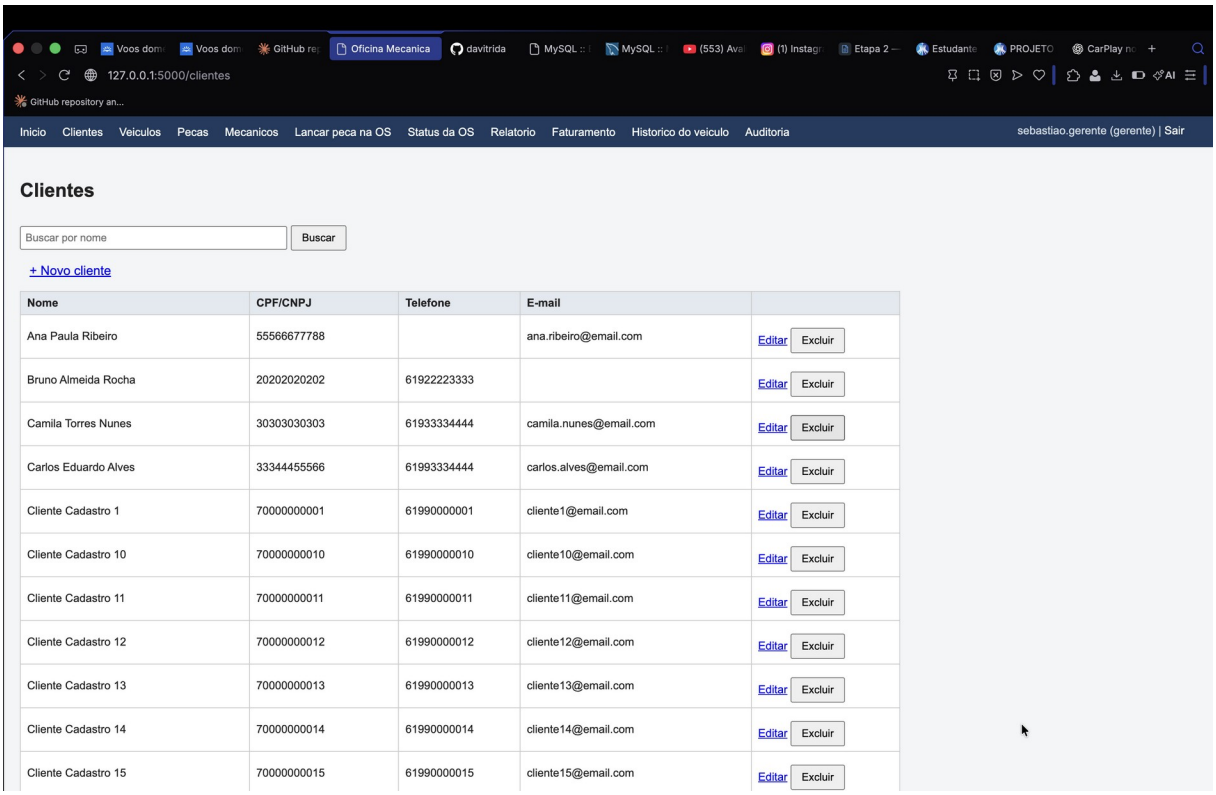


Figura 2 — Lista de clientes (CRUD).

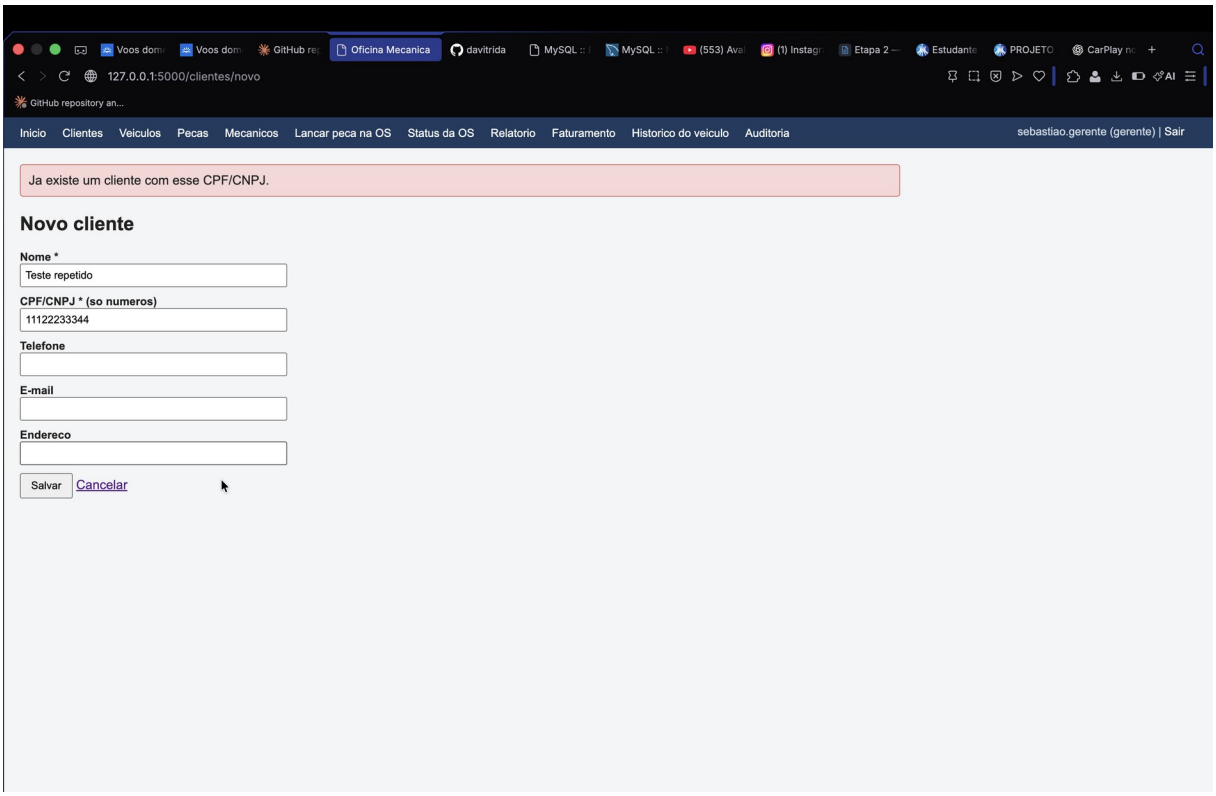


Figura 3 — CPF repetido: erro do banco traduzido para português.

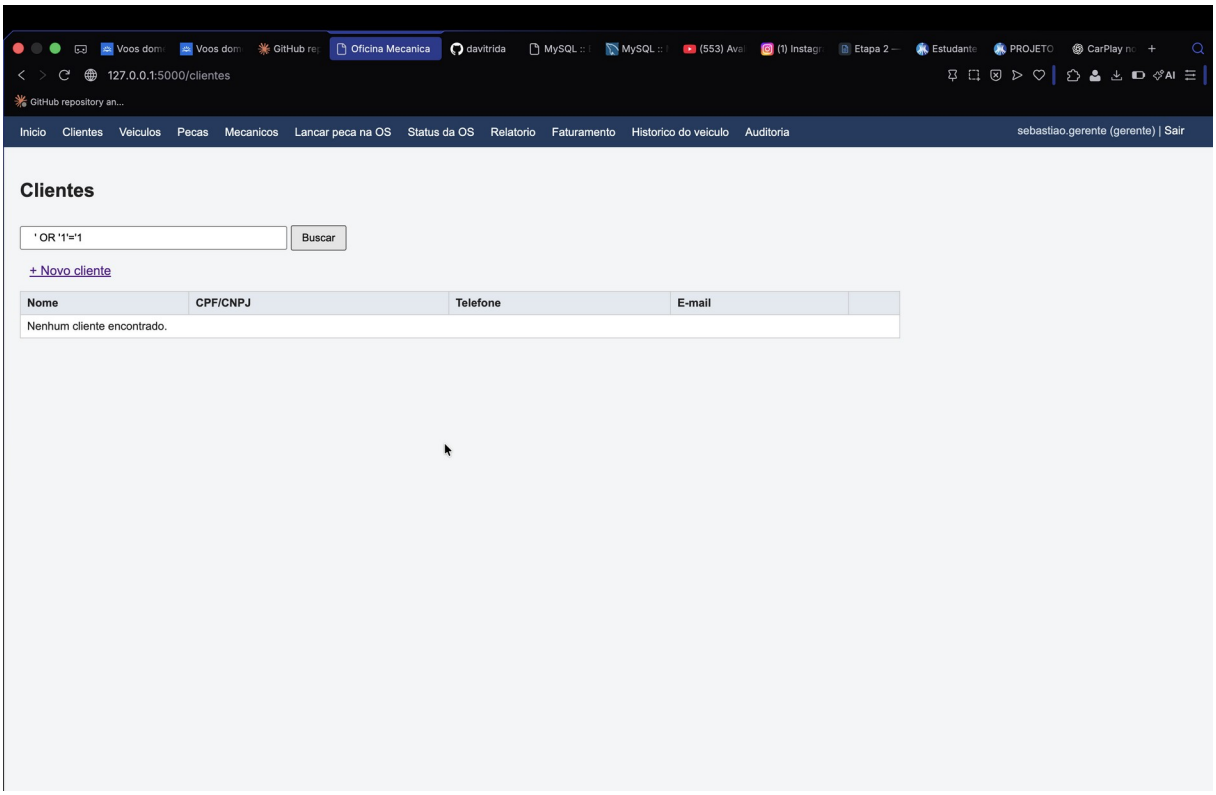


Figura 4 — Teste de injeção de SQL: busca com ' OR '1'='1 sem resultados.

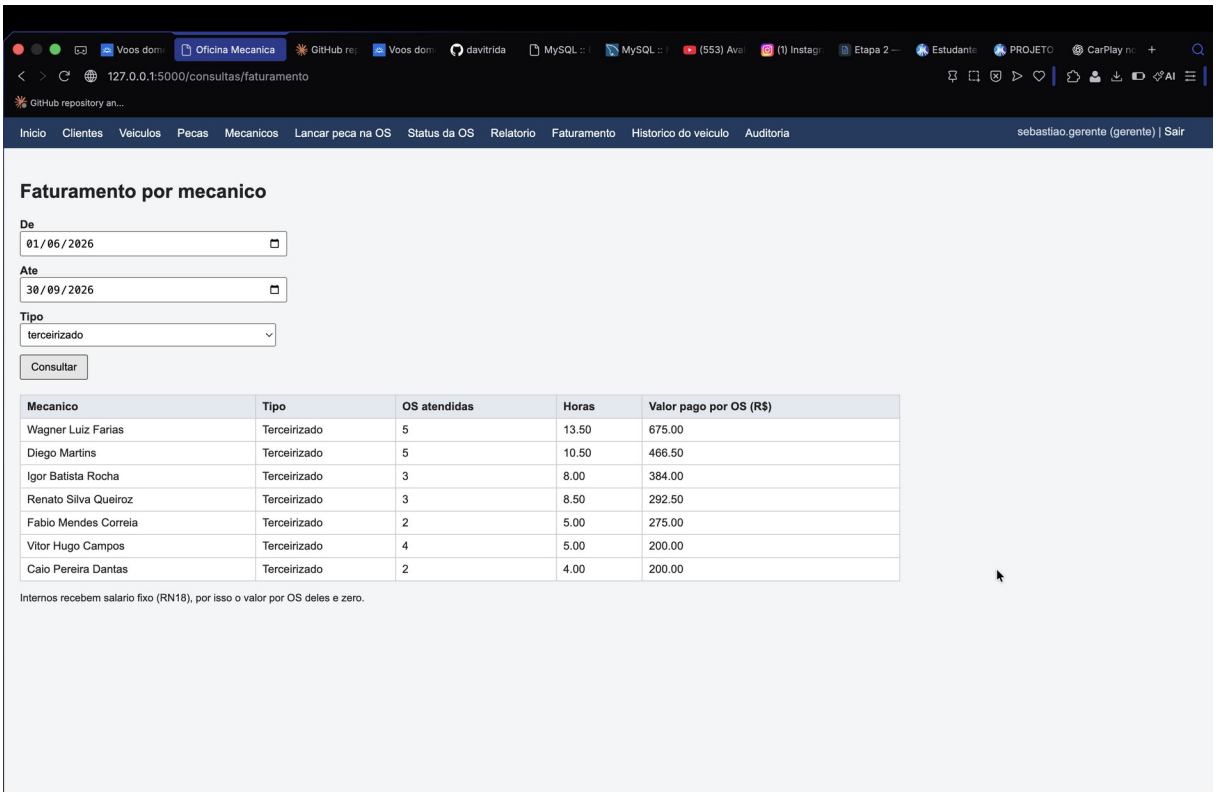


Figura 5 — Consulta parametrizada 1: faturamento por mecânico.

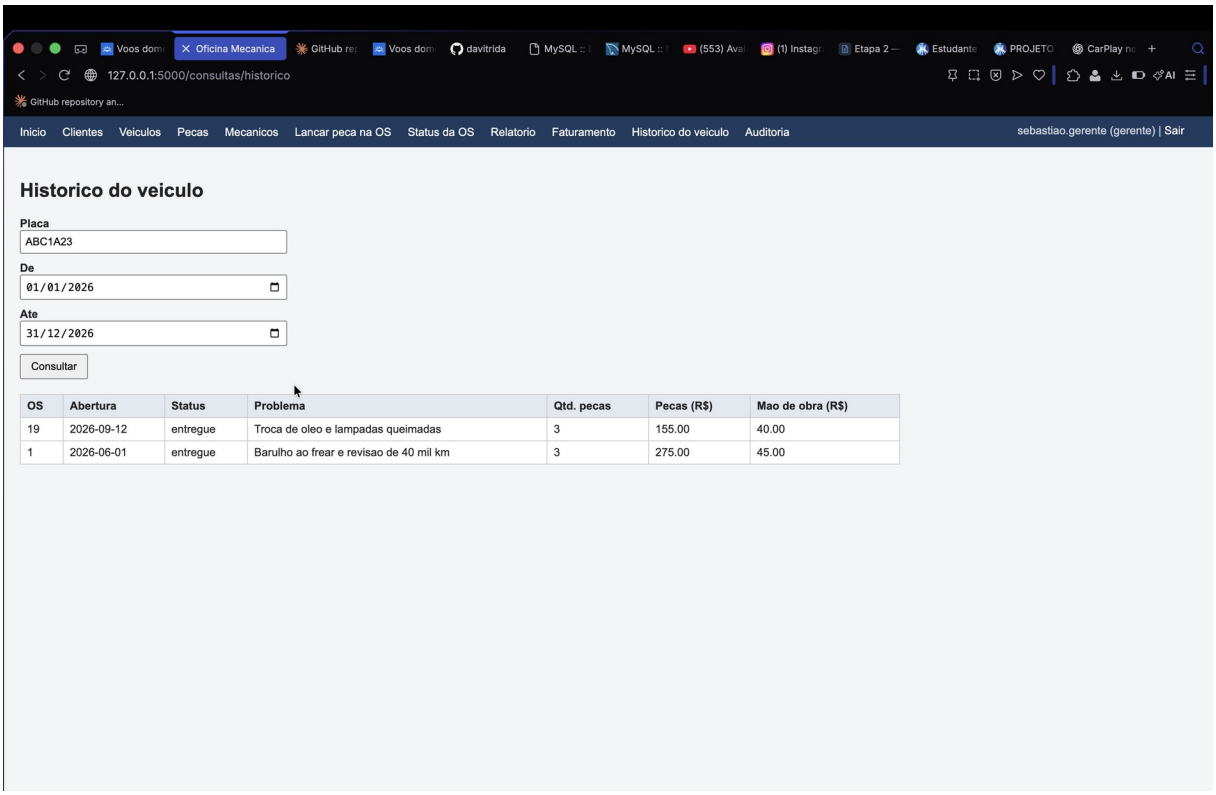


Figura 6 — Consulta parametrizada 2: histórico do veículo ABC1A23.

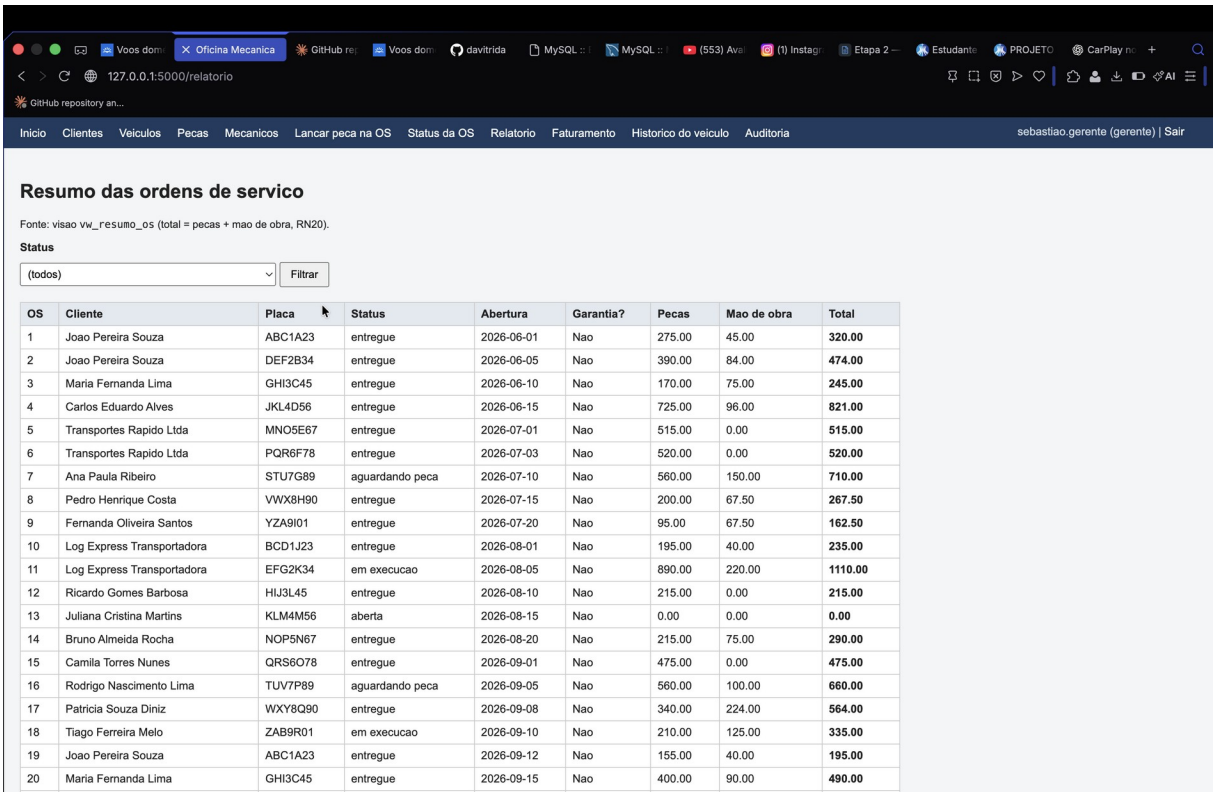


Figura 7 — Relatório alimentado pela visão vw_resumo_os.

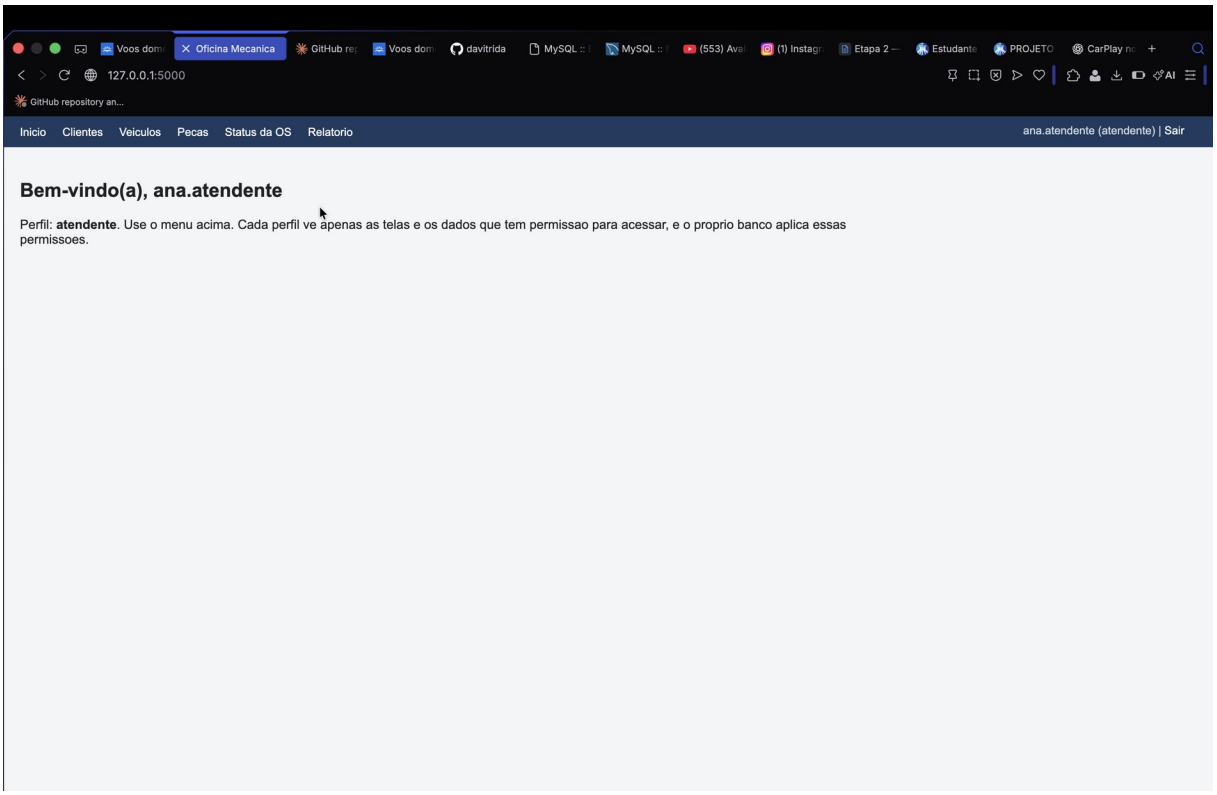


Figura 8 — Menu do perfil atendente.

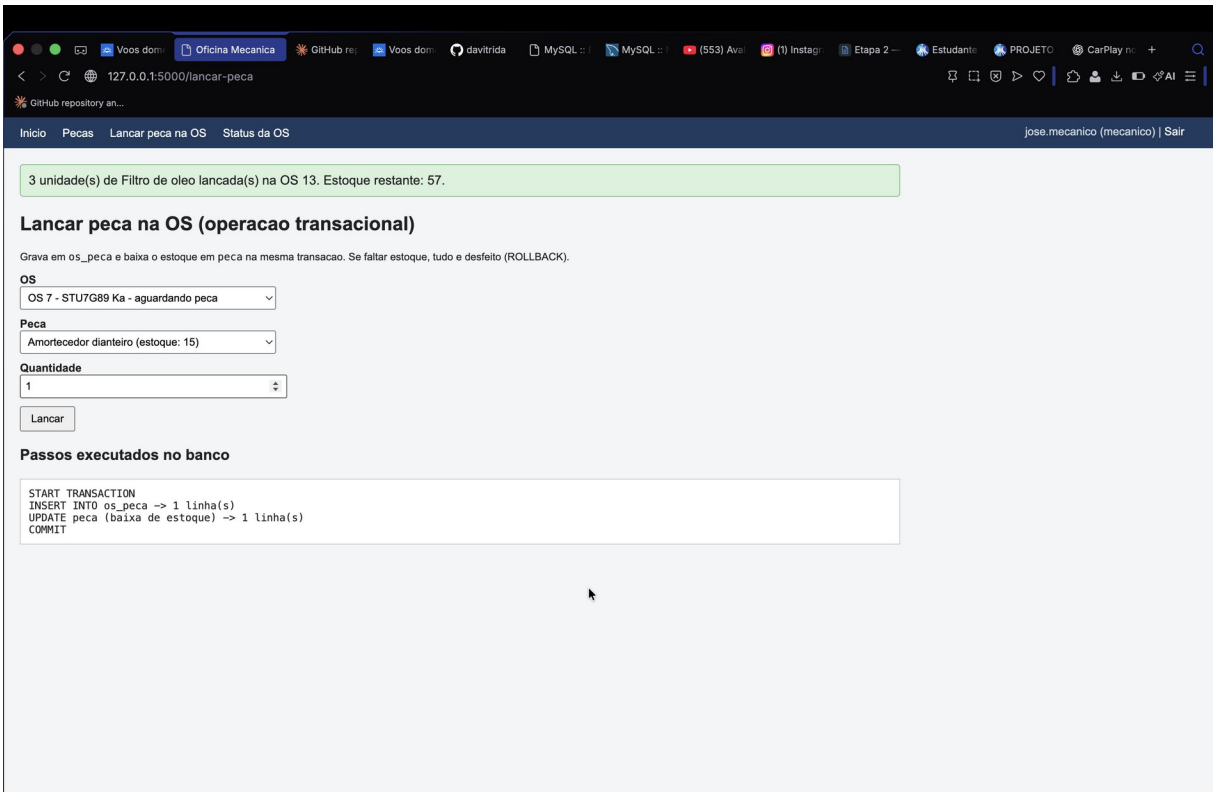


Figura 9 — Lançamento com sucesso: START TRANSACTION, INSERT, UPDATE e COMMIT.

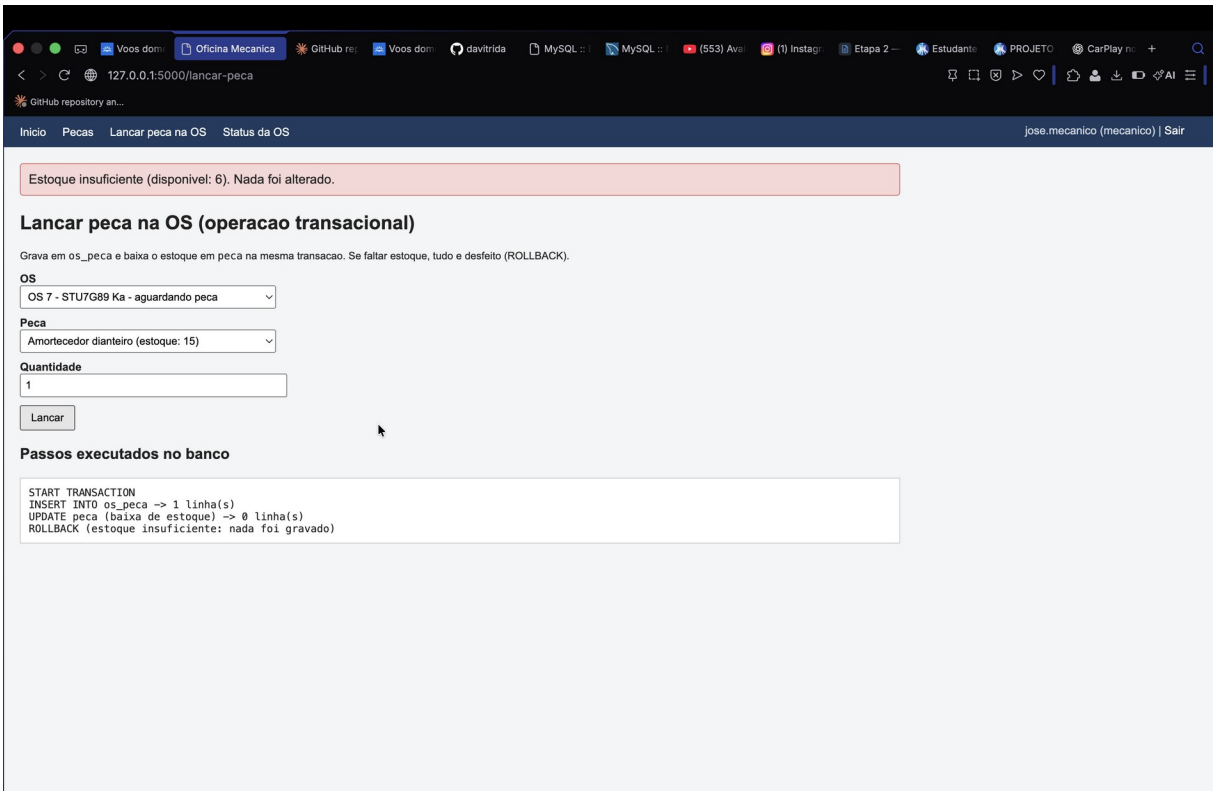


Figura 10 — Lançamento com falha provocada: UPDATE sem linhas e ROLLBACK.

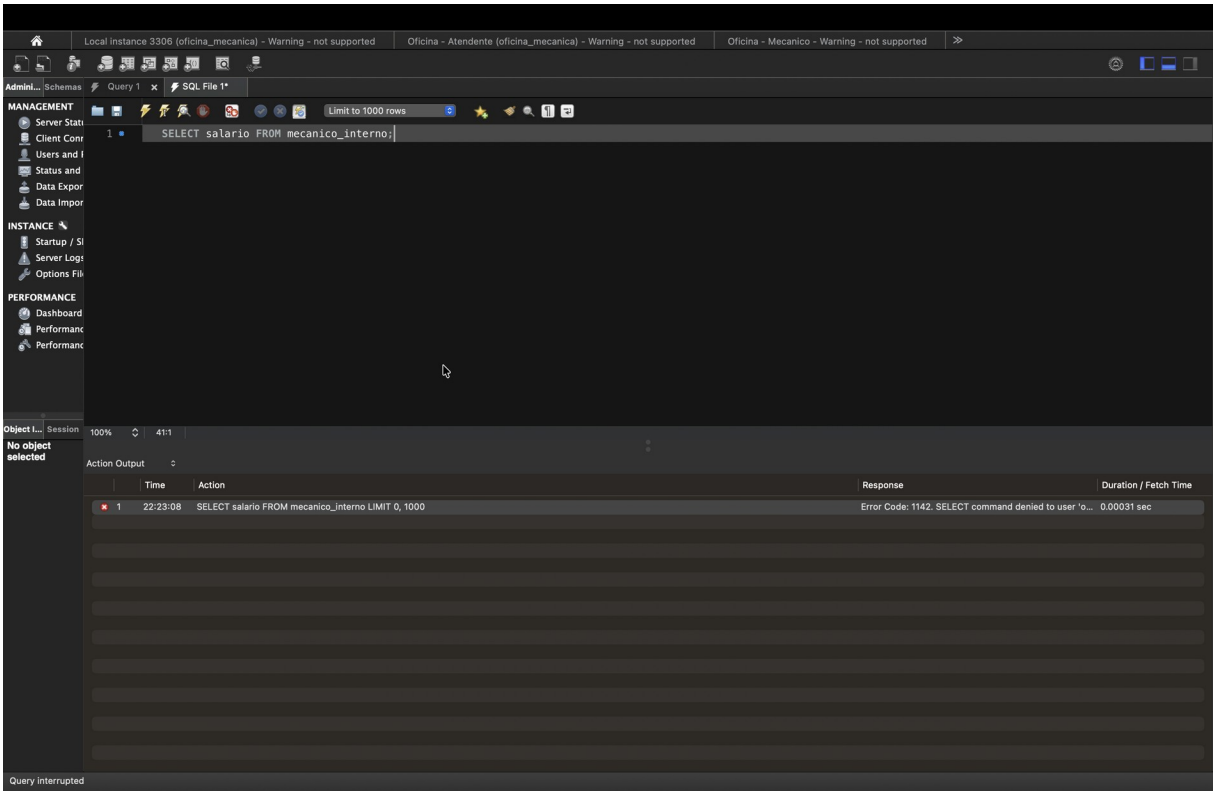


Figura 11 — Atendente recusado ao ler salário (erro 1142).

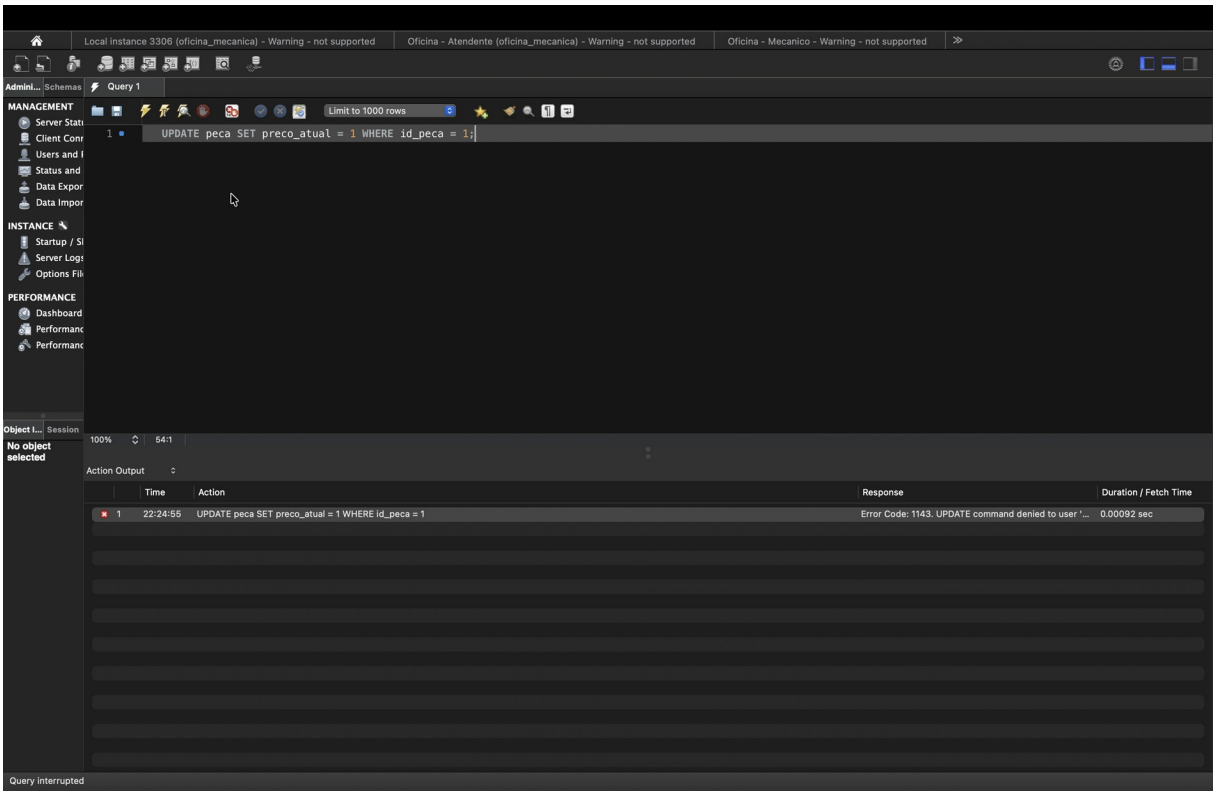


Figura 12 — Mecânico recusado ao mudar preço (erro 1143, privilégio por coluna).

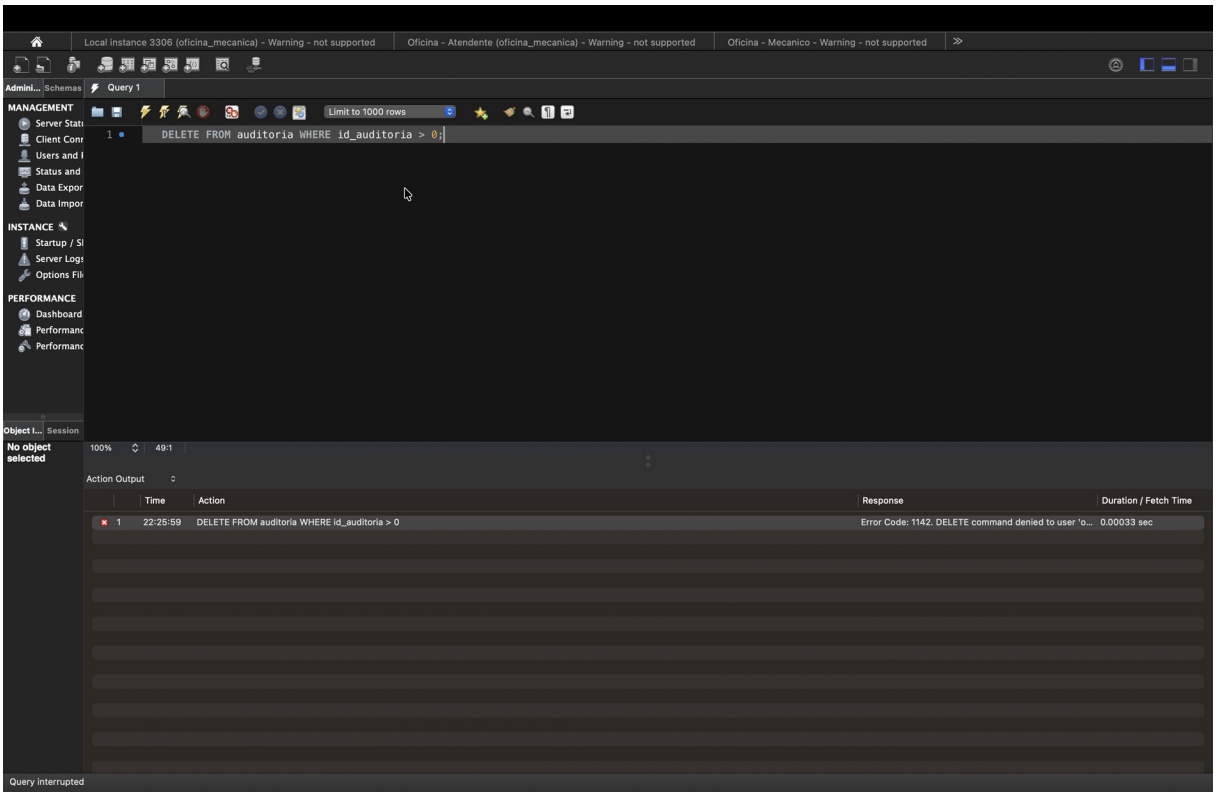


Figura 13 — Gerente recusado ao apagar a auditoria (erro 1142).

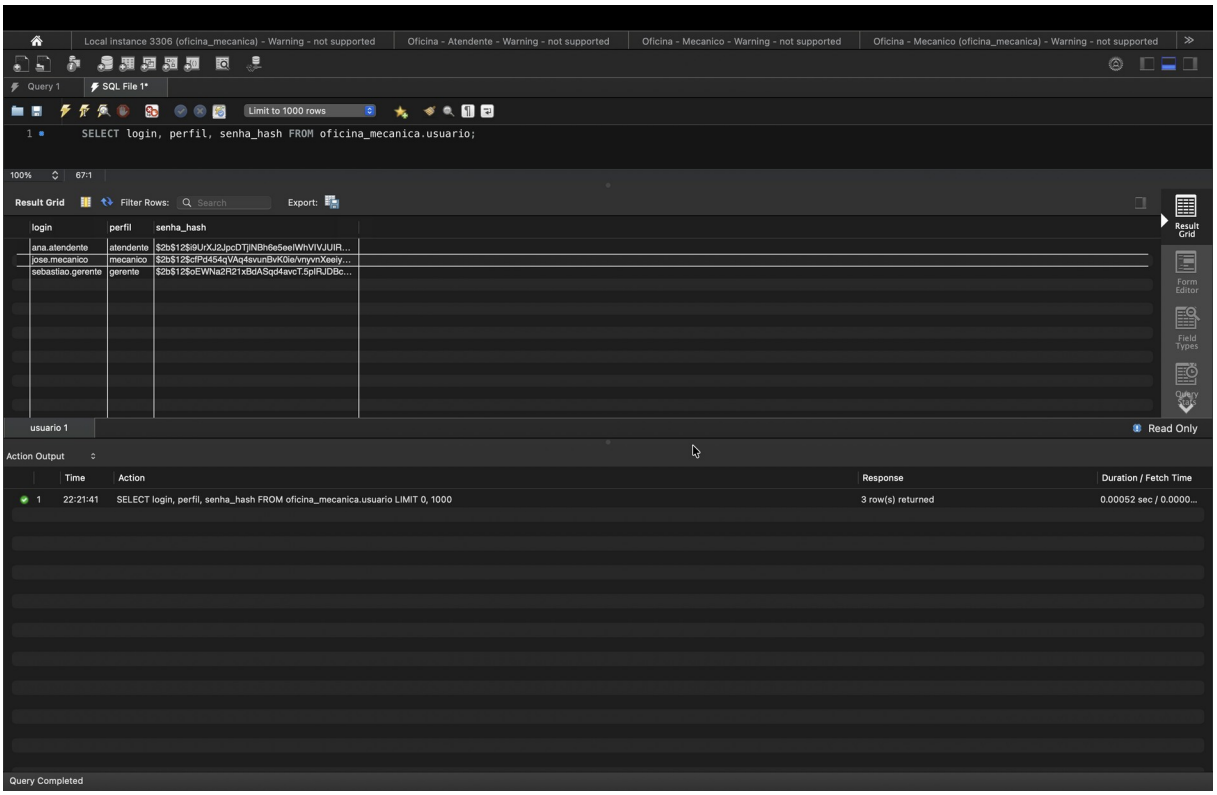


Figura 14 — Tabela usuario com senhas guardadas em hash bcrypt.

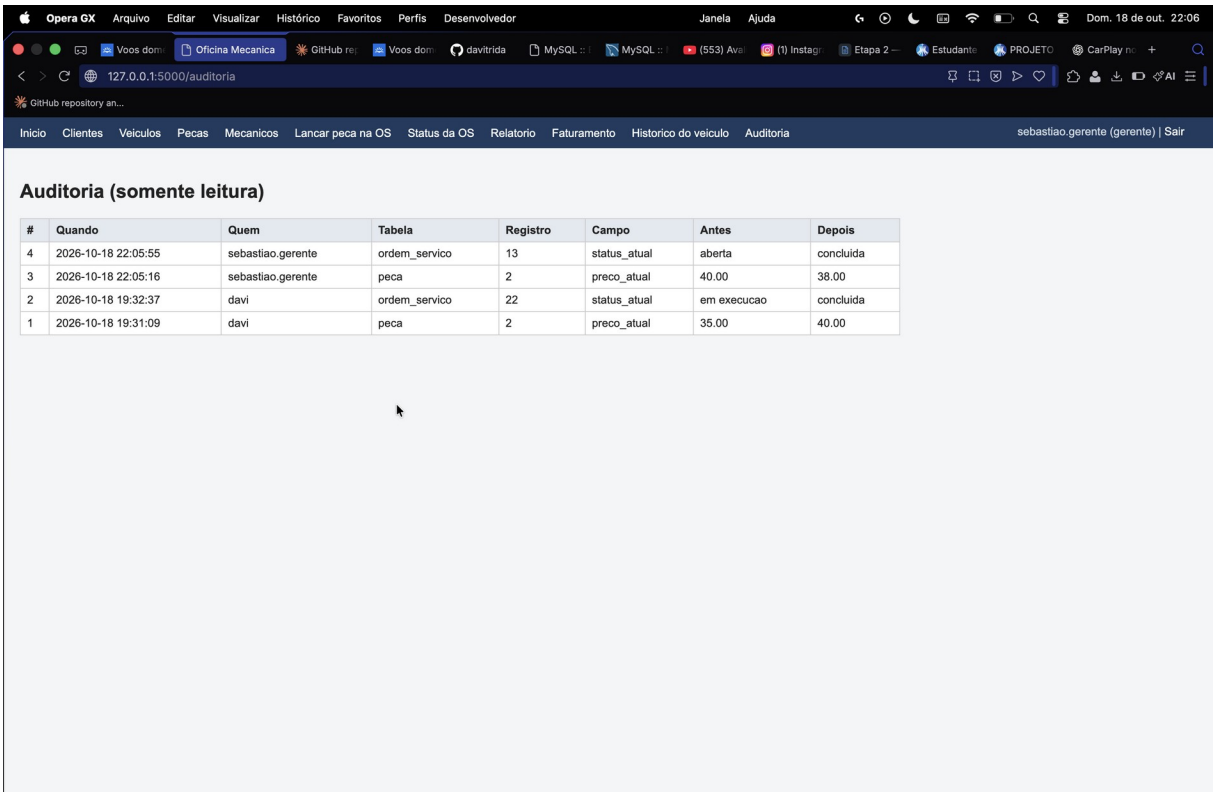


Figura 15 — Tela de auditoria preenchida pelos gatilhos.